PROJECT REPORT

AI-Based Student Feedback Sentiment Analyzer

NLP · Machine Learning · Naive Bayes · Python

Technology: Python | Scikit-learn | NLP | Naive Bayes

Domain: Artificial Intelligence & Machine Learning

Institution: VIT Bhopal University

Programme: B.Tech — Computer Science (AI & ML)

Year: 2026

01 Project Overview

An AI-Based Student Feedback Sentiment Analyzer that automatically classifies student feedback into Positive, Negative, or Neutral using NLP and Naive Bayes ML. It detects specific issues (Speed, Difficulty, Clarity) in negative feedback and generates actionable improvement suggestions for educators. Supports both free-text and predefined input modes with automatic model training.

Algorithm: Multinomial Naive Bayes

Feature Extraction: Bag-of-Words (CountVectorizer)

Sentiment Classes: Positive, Negative, Neutral

Issue Detection: Speed · Difficulty · Clarity

Input Modes: Custom text + Predefined options

Dependencies: Python, Pandas, Scikit-learn

02 Introduction

Educational institutions collect large volumes of student feedback but lack the capacity to process it efficiently. Manual review is slow, subjective, and often results in valuable insights being lost. This project applies NLP and Machine Learning to automate sentiment analysis of student feedback, providing instant structured insights to educators without manual review.

For students in AI & ML, building this system from scratch offers hands-on experience with the complete ML pipeline — from data preparation and feature engineering through model training, evaluation, and deployment.

03 Problem Statement

Volume: Hundreds of responses cannot be manually read per course in a timely manner.

Subjectivity: Human reviewers interpret the same feedback inconsistently.

Actionability: The link between student comments and specific improvements is rarely made explicit.

Timeliness: By the time feedback is reviewed manually, the course may already be over.

04 Objectives

Build a working sentiment classifier (Positive / Negative / Neutral) using Naive Bayes ML.

Detect issue categories (Speed, Difficulty, Clarity) in negative feedback and generate targeted suggestions.

Provide a CLI interface supporting both free-text input and predefined feedback selection.

Demonstrate the full ML pipeline: data prep → feature extraction → training → inference.

Produce extensible, well-structured code ready for future GUI or web-app integration.

05 System Architecture

The system follows a five-stage modular pipeline from raw text input to structured output:

Stage 1 — Input Layer: User enters text or selects from predefined options via CLI (app.py).

Stage 2 — Preprocessing: CountVectorizer tokenises and converts text to Bag-of-Words feature vector.

Stage 3 — ML Inference: Trained Naive Bayes model predicts: Positive, Negative, or Neutral.

Stage 4 — Issue Detection: Negative predictions trigger keyword scan for Speed, Difficulty, Clarity issues.

Stage 5 — Output: System prints sentiment label, detected issues, and tailored suggestions.

06 Technologies and Tools

Python 3.8+: Core programming language

Pandas: Data loading and CSV handling

Scikit-learn: ML pipeline: vectorisation, training, prediction

CountVectorizer (Scikit-learn): Text-to-numeric Bag-of-Words feature extraction

MultinomialNB (Scikit-learn): Probabilistic Naive Bayes text classifier

pickle / joblib: Model serialisation and persistence to disk

feedback.csv (custom): Labelled training dataset

CLI (app.py): Command-line user interface

07 Dataset Description

Custom-curated feedback.csv with two columns: feedback (raw text) and sentiment (label: Positive / Negative / Neutral). Sample entries:

Positive: "The professor explained every concept clearly. Very engaging lessons."

Negative: "The teacher goes too fast and the topics are very difficult to follow."

Neutral: "The class was okay. Some topics were interesting, others were not."

08 Methodology

Step 1 — Data Collection & Preparation

Curated a labelled dataset of student feedback samples stored in feedback.csv. Each entry contains a text string and a sentiment label. Data was checked for completeness and vocabulary diversity across all three sentiment classes.

Step 2 — Text Preprocessing & Feature Extraction

Raw text is transformed into numerical feature vectors using CountVectorizer (Bag-of-Words). The vectorizer builds a vocabulary from the training corpus and represents each document as a word-frequency vector. Example:

Vocab: {clear:0, fast:1, difficult:2 ...} | "Too fast and difficult" → [0, 1, 1, ...]

Step 3 — Model Selection: Naive Bayes

Multinomial Naive Bayes was chosen for text classification because it is purpose-built for discrete word-count features, trains rapidly on small datasets, performs well in practice despite its independence assumption, and is highly interpretable for academic demonstration.

Step 4 — Training & Model Persistence

train.py fits the CountVectorizer on the training corpus, transforms the text to feature vectors, and trains the Naive Bayes classifier. Both objects are serialised to disk using pickle/joblib. app.py loads them on startup — avoiding retraining on every run.

Step 5 — Issue Detection (Rule-Based Layer)

When a feedback item is classified as Negative, a secondary keyword scan identifies specific issues:

Speed — Trigger Keywords: fast, slow, pace, rushed — Example Suggestion: Adjust lecture pace; provide recordings.

Difficulty — Trigger Keywords: difficult, hard, confusing, complex — Example Suggestion: Include more examples; simplify explanations.

Clarity — Trigger Keywords: unclear, vague, not clear, hard to follow — Example Suggestion: Use visual aids; clarify core concepts.

09 Project Structure

```
AIML PROJECTS/
 ├── data/feedback.csv       # Labelled training dataset
 ├── train.py                # Model training script
 ├── app.py                  # Main application (inference + CLI)
 ├── README.md               # Documentation
 ├── report.pdf              # This report
 └── .gitignore              # Excludes binaries from version control
```

10 Key Features

Sentiment Classification — Classifies any feedback as Positive, Negative, or Neutral using a trained Naive Bayes model with probabilistic confidence scoring.

Dual Input Mode — Supports free-text custom input and predefined option selection — flexible for both testing and live demonstration.

Issue Detection — Scans negative feedback for Speed, Difficulty, and Clarity issues. Multiple issues can be flagged per item.

Actionable Suggestions — Generates specific, issue-matched improvement recommendations — not generic advice.

Auto Model Training — If no saved model is found, training runs automatically before inference. Fully self-contained.

Lightweight & Portable — Only two external dependencies. No GPU, no internet. Runs on any student laptop.

11 How to Run

Step 1 — Install dependencies:

pip install pandas scikit-learn

Step 2 — Run the application:

python app.py

Note: If no trained model exists on disk, the system automatically runs train.py before launching. No manual training step is needed.

13 Concepts Covered

Natural Language Processing (NLP): Enables computers to understand text. Used here for tokenisation, vocabulary building, and feature extraction.

ML Classification: Supervised learning predicting class labels from examples. This project implements three-class text classification.

Naive Bayes Algorithm: Probabilistic classifier using Bayes' theorem. Fast, effective on small text datasets, and highly interpretable.

Bag-of-Words: Represents text as word frequency vectors. CountVectorizer automates vocabulary building and transformation.

Model Persistence: Saving trained models to disk with pickle/joblib for reuse without retraining — essential for real deployments.

Hybrid AI (ML + Rules): Combines statistical ML for sentiment classification with deterministic keyword rules for issue detection.

14 Applications

Educational Quality Improvement: Auto-process end-of-semester surveys and give faculty structured sentiment summaries with issue breakdowns.

Real-Time Feedback: Integrated with LMS platforms (Moodle, Google Classroom) to give instructors insights before the next class.

Accreditation Support: Generate quantitative student satisfaction metrics for institutional quality assurance and accreditation reports.

NLP/ML Education Tool: A concrete, working demonstration of the complete ML pipeline ideal for introductory AI/ML courses.

Scalable Analytics: Extend to MOOCs, online tutoring platforms, and corporate training with minor dataset modifications.

16 Future Improvements

Advanced NLP Models: Replace Naive Bayes with transformer models (BERT, DistilBERT) for contextual understanding, negation handling, and sarcasm detection.

Larger Dataset: Expand to thousands of real anonymised feedback samples across multiple institutions, subjects, and academic levels.

Web Application: Build a Flask/FastAPI web app with bulk CSV upload, dashboard analytics, and exportable sentiment reports.

Multi-Language Support: Add support for Indian regional languages (Hindi, Tamil, etc.) using multilingual NLP models.

Aspect-Based Sentiment: Move from three-class to aspect-level analysis — separately scoring Content, Delivery, Pacing, and Assessment quality.

Visualisation Dashboard: Integrate Plotly/Matplotlib charts showing sentiment trends over time and per-course issue frequency heatmaps.

17 Conclusion

The AI-Based Student Feedback Sentiment Analyzer successfully demonstrates how foundational NLP and Machine Learning techniques can be combined to solve a real educational problem. The system classifies feedback accurately, detects specific issues, and provides actionable suggestions — all within a lightweight, self-contained Python application.

The project achieves all primary objectives: three-class sentiment classification, issue detection, suggestion generation, and a dual-mode CLI interface. From a learning standpoint, it provides hands-on experience with the complete ML pipeline and highlights the value of combining statistical models with rule-based systems — a key design pattern in applied AI.

The modular architecture is well-suited for future extension: swapping in transformer models, expanding the dataset, or wrapping the backend in a web application are all straightforward next steps. This project demonstrates that even with foundational tools, it is possible to build a functional, intelligent system that delivers real educational value.